contractSpy

Decentralised Smart Contract Security & Threat Intelligence on BOT Chain

Version 1.0 Date: November 2026 Network: BOT Chain (ID: 677)

Abstract

As decentralised finance (DeFi) and web3 applications continue to scale, the attack surface for malicious actors expands proportionally. Smart contracts, while immutable and transparent, are frequently targeted by exploits, unauthorised upgrades, and insider threats. **contractSpy** is a decentralised, fully on-chain threat intelligence and smart contract monitoring platform built exclusively for the BOT Chain ecosystem. By removing reliance on centralised backend infrastructure, contractSpy allows users to register contracts, configure custom threat-detection parameters, and record security alerts immutably on-chain. This whitepaper outlines the architecture, core feature set, and operational workflow of the contractSpy protocol.

1. Introduction

The traditional approach to smart contract monitoring relies heavily on centralised Software-as-a-Service (SaaS) platforms. These platforms observe blockchain states and push notifications via Web2 channels (email, SMS, Telegram). While effective, they introduce central points of failure, require recurring fiat subscriptions, and lack transparency regarding the historical accuracy of their alert logs.

contractSpy solves this by decentralising the threat intelligence workflow. Operating directly on BOT Chain (Chain ID: 677), the protocol leverages the user's local client to poll RPC endpoints based on on-chain configurations. When a threat is detected, the alert is committed directly to the blockchain, creating a permanent, tamper-proof audit trail of security events.

2. Core Architecture

The contractSpy platform operates on a hybrid model that maximises decentralisation while maintaining high performance:

- **The Smart Contract Registry:** A monolithic Solidity contract deployed on BOT Chain. It acts as the central state machine, storing user profiles, registered monitored contracts, bespoke monitoring rules, immutable alert records, and community-submitted security reports.
- **The Decentralised Monitoring Engine:** Instead of a centralised backend server polling the blockchain, the contractSpy frontend acts as a local monitoring node. When a user connects their wallet and keeps the dashboard open, their client continuously queries the BOT Chain RPC for events matching their on-chain configuration.
- **Immutable Alerting:** Upon detecting an anomaly (e.g., a large withdrawal or unauthorised admin change), the client prompts the user to submit an on-chain transaction that records the alert in the smart contract.

3. Key Features & Threat Detection

contractSpy provides comprehensive visibility into contract states and token movements. Users can configure up to 50 smart contracts per wallet, tuning the following detection modules:

Detection Module	Description	Severity Level
Ownership & Admin Changes	Monitors `OwnershipTransferred` and `RoleGranted` events to detect potentially hostile takeovers or compromised private keys.	Critical
Contract Upgrades	Tracks `Upgraded` events in proxy patterns to alert users of underlying logic changes.	Critical
Unexpected Minting	Detects token transfers originating from the zero address (`0x0...0`) to flag infinite mint glitches or unauthorised issuance.	Medium
Large Withdrawals	Monitors specific `Withdrawal` events or token transfers exceeding a user-defined threshold (e.g., > 1 ETH equivalent).	Medium
Abnormal Activity	Flags high-frequency transaction patterns within short block intervals, often indicative of an ongoing exploit or flash loan attack.	Low / Medium

> **Community Threat Intelligence:** Beyond automated alerts, contractSpy includes a crowdsourced Security Report system. Users can submit detailed vulnerability reports directly to the blockchain, which can be upvoted by the community, creating a decentralised database of known threats.

4. Technical Implementation

The protocol is governed by a highly gas-optimised smart contract structure. Core data components are segmented into distinct structs to optimise storage and retrieval:

- `UserProfile`: Maintains user state, total monitored contracts, and premium status flags.
- `MonitoredContract` & `MonitoringConfig`: Stores the target address and boolean flags for each detection module, alongside specific numeric thresholds (e.g., `largeWithdrawalThreshold`).
- `SecurityAlert`: Records the exact transaction hash, description, severity (1=Low, 2=Medium, 3=Critical), and acknowledgment status of a detected event.

4.1 Access Control and Security

The smart contract inherits standard `ReentrancyGuard` and `Ownable` patterns. Strict modifiers (`userExists`, `contractOwner`) ensure that only the verified owner of a monitored contract profile can update configurations, log alerts, or acknowledge resolved threats.

5. User Operational Workflow

The typical lifecycle for a security analyst or protocol operator using contractSpy is completely frictionless and non-custodial:

1. **Connect & Register:** The user connects a Web3 wallet (e.g., MetaMask) to the BOT Chain network and pays a small gas fee to register an on-chain profile.
2. **Add Contracts:** The user inputs the address of a target contract (e.g., a DeFi liquidity pool or a DAO treasury) and configures the monitoring parameters.
3. **Active Monitoring:** The user leaves the contractSpy interface active. The system silently polls the RPC every 60 seconds against the latest block headers.
4. **Alert Acknowledgment:** If an exploit signature is matched, an on-chain alert is created. The user investigates the provided transaction hash via the BOT Chain Explorer and acknowledges the alert on-chain to resolve it.

6. Conclusion

By migrating security monitoring from opaque, centralised servers to a transparent, user-driven protocol, contractSpy sets a new standard for Web3 threat intelligence. Built natively for the BOT Chain ecosystem, it empowers developers, auditors, and everyday users to take proactive, verifiable control over the security of their digital assets and smart contracts.